AMRUTA INSTITUTE OF ENGINEERING AND MANAGEMENT SCIENCES – POLYTECHNIC

Course: Database System Concepts and PL/SQL

Course Code: 20CS34P

Week 11

PL/SQL

PL/SQL Introduction

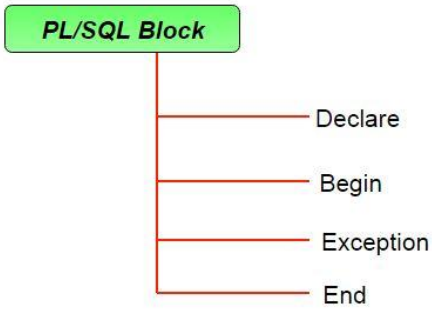
PL/SQL Introduction

- PL/SQL is a block structured language that combine the power of SQL with procedural statements.
- All the statements of a block are passed to oracle engine all at once which increases processing speed and decreases the traffic.

Features of PL/SQL:

1. PL/SQL is basically a procedural language, which provides the functionality of decision making, iteration and many more features of procedural programming languages.
2. PL/SQL can execute a number of queries in one block using single command.
3. PL/SQL provides a feature to handle the exception which occurs in PL/SQL block known as exception handling block.
4. PL/SQL Offers extensive error checking.
5. PL/SQL is case insensitive.

Structure of PL/SQL Block:



A block has the following structure:

```
DECLARE
   declaration statements;

BEGIN
   executable statements

EXCEPTIONS
   exception handling statements
END;
```

- Declare section starts with **DECLARE** keyword in which variables, constants, records as cursors can be declared which stores data temporarily. It basically consists definition of PL/SQL identifiers. This part of the code is optional.
- Execution section starts with **BEGIN** and ends with **END** keyword. This is a mandatory section and here the program logic is written to perform any task like loops and conditional statements. It supports all DML commands, DDL commands and SQL*PLUS built-in functions as well.
- Exception section starts with **EXCEPTION** keyword. This section is optional which contains statements that are executed when a run-time error occurs.

PL/SQL Variables

You must declare the PL/SQL variable in the declaration section or in a package as a global variable. After the declaration, PL/SQL allocates memory for the variable's value and the storage location is identified by the variable name.

1) Syntax for declaring variable:

variable_name datatype [NOT NULL := value];

Example:

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
        var1 INTEGER;
        var2 REAL;
        var3 varchar2(20) ;
 BEGIN
        null;
END;
/
```

1) INITIALISING VARIABES:

The variables can also be initialised just like in other programming languages. Let us see an example for the same:

```
SQL> SET SERVEROUTPUT ON;

SQL> DECLARE

    var1 INTEGER := 2 ;

    var3 varchar2(20) := 'I Love GeeksForGeeks' ;

  BEGIN

    null;

  END;

  /
```

3) **Displaying Output**

The outputs are displayed by using DBMS_OUTPUT which is a built-in package that enables the user to display output, debugging information, and send messages from PL/SQL blocks, subprograms, packages, and triggers.

Let us see an example to see how to display a message using PL/SQL :

```
SQL> SET SERVEROUTPUT ON;

SQL> DECLARE

    var varchar2(40) := 'I love GeeksForGeeks' ;
```

```
BEGIN
  dbms_output.put_line(var);
END;
/
```

PL/SQL Datatypes

A wide range of data types are supported and can be used to declare variables in a PL/SQL block. Table summarizes some datatypes.

Datatype	Description
CHAR (*size*)	Fixed-length character data of length *size* bytes.
VARCHAR2 (*size*)	Variable-length character data.
NCHAR(*size*)	Fixed-length character data of length *size* characters or bytes, depending on the national character set.
NVARCHAR2 (*size*)	Variable-length character data of length *size* characters or bytes, depending on national character set. A maximum *size* must be specified.
CLOB	Single-byte character data.
NCLOB	Single-byte or fixed-length multibyte national character set (NCHAR) data.
LONG	Variable-length character data.
NUMBER (p, s)	Variable-length numeric data. Maximum precision *p* and/or scale *s* is 38.
DATE	Fixed-length date and time data, ranging from Jan. 1, 4712 B.C.E. to Dec. 31, 4712 C.E.
BLOB	Unstructured binary data.
BFILE	Binary data stored in an external file.

RAW (size)	Variable-length raw binary data.
LONG RAW	Variable-length raw binary data.
ROWID	Binary data representing row addresses.
MLSLABEL	Trusted Oracle datatype.

DECLARE

Code Snippet with numeric data types:

```
numA INTEGER := 10;
  numB NUMBER(10) := 10.8;
  numC DOUBLE PRECISION := 10.6;
BEGIN
  dbms_output.put_line('The numA is: ' || numA);
  dbms_output.put_line('The numB is: ' || numB);
  dbms_output.put_line('The numC is: ' || numC);
END;
/
```

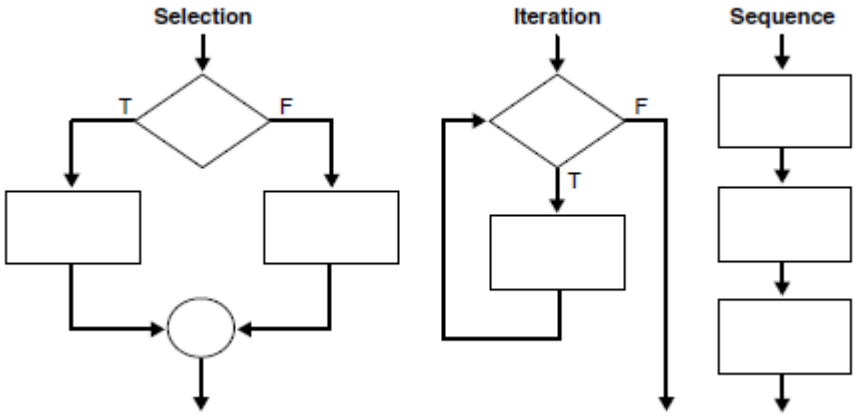
In the above code, we have INTEGER, NUMBER, and DOUBLE PRECISION data types with the variables numA, numB and numC respectively. numB has a number data type with precision 10 and has no digits after the decimal.

Code Implementation with some data types:

```
DECLARE
   m CHAR(20) := 'softwareTest!';
   n VARCHAR2(30) := 'plsql';
   o NCHAR(30) := 'plsql datatypes';
   p NVARCHAR2(30) := 'plsql literals';
   presentDt DATE:= SYSDATE;
   a INTEGER := 16;
   b NUMBER(20) := 11.2;
   c DOUBLE PRECISION := 14.7;
BEGIN
```

```
  dbms_output.put_line('The char datatype is: ' || m);
  dbms_output.put_line('The varchar datatype is: ' || n);
  dbms_output.put_line('The nchar datatype is: ' || o);
  dbms_output.put_line('The nvarchar2 datatype is: ' || p);
  dbms_output.put_line('The current date is: ' || presentDt);
  dbms_output.put_line('The number a is: ' || a);
  dbms_output.put_line('The number b is: ' || b);
  dbms_output.put_line('The number c is: ' || c);
END;
/
```

Control Structures in PL/SQL



- The selection structure tests a condition, then executes one sequence of statements instead of another, depending on whether the condition is true or false. A condition is any variable or expression that returns a BOOLEAN value (TRUE or FALSE).
- The iteration structure executes a sequence of statements repeatedly as long as a condition holds true.
- The sequence-structure simply executes a sequence of statements in the order in which they occur.

- Using the IF-THEN Statement

The simplest form of IF statement associates a condition with a sequence of statements enclosed by the keywords THEN and END IF (not ENDIF)

The sequence of statements is executed only if the condition is TRUE. If the condition is FALSE or NULL, the IF statement does nothing. In either case, control passes to the next statement.

Example: Using a Simple IF-THEN Statement

```
DECLARE
sales NUMBER(8,2) := 10100;
quota NUMBER(8,2) := 10000;
bonus NUMBER(6,2);
emp_id NUMBER(6) := 120;
BEGIN
IF sales > (quota + 200) THEN
bonus := (sales - quota)/4;
UPDATE employees SET salary = salary + bonus WHERE employee_id = emp_id;
END IF;
END;
/
```

Using CASE Statements

Like the IF statement, the CASE statement selects one sequence of statements to execute. However, to select the sequence, the CASE statement uses a selector rather than multiple Boolean expressions. A selector is an expression whose value is used to select one of several alternatives.

Example: Using the CASE-WHEN Statement

```
DECLARE
grade CHAR(1);
BEGIN
grade := 'B';
CASE grade
WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');
WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Very Good');
```

```
WHEN 'C' THEN DBMS_OUTPUT.PUT_LINE('Good');
WHEN 'D' THEN DBMS_OUTPUT.PUT_LINE('Fair');
WHEN 'F' THEN DBMS_OUTPUT.PUT_LINE('Poor');
ELSE DBMS_OUTPUT.PUT_LINE('No such grade');
END CASE;
END;
/
```

Controlling Loop Iterations: LOOP and EXIT Statements

LOOP statements execute a sequence of statements multiple times. There are three forms of LOOP statements: LOOP, WHILE-LOOP, and FOR-LOOP.

Using the LOOP Statement

The simplest form of LOOP statement is the basic loop, which encloses a sequence of statements between the keywords LOOP and END LOOP, as follows:

```
LOOP
sequence_of_statements
END LOOP;
```

Using the WHILE-LOOP Statement

The WHILE-LOOP statement executes the statements in the loop body as long as a condition is true:

```
WHILE condition LOOP
sequence_of_statements
END LOOP;
```

Using the FOR-LOOP Statement

Simple FOR loops iterate over a specified range of integers. The number of iterations is known before the loop is entered. A double dot (..) serves as the range operator. The range is evaluated when the FOR loop is first entered and is never re-evaluated. If the lower bound equals the higher bound, the loop body is executed once.

Example: Using a Simple FOR LOOP Statement

```
DECLARE
p NUMBER := 0;
BEGIN
FOR k IN 1..500 LOOP -- calculate pi with 500 terms
p := p + ( ( (-1) ** (k + 1) ) / ((2 * k) - 1) );
END LOOP;
p := 4 * p;
DBMS_OUTPUT.PUT_LINE( 'pi is approximately : ' || p ); -- print result
END;
/
```

What is Procedure in PL/SQL?

A **Procedure** in PL/SQL is a subprogram unit that consists of a group of PL/SQL statements that can be called by name. Each procedure in PL/SQL has its own unique name by which it can be referred to and called. This subprogram unit in the Oracle database is stored as a database object.

.

Below are the characteristics of Procedure subprogram unit in PL/SQL:

- Procedures are standalone blocks of a program that can be stored in the database.
- Call to these PLSQL procedures can be made by referring to their name, to execute the PL/SQL statements.
- It is mainly used to execute a process in PL/SQL.
- It can have nested blocks, or it can be defined and nested inside the other blocks or packages.
- It contains declaration part (optional), execution part, exception handling part (optional).
- The values can be passed into Oracle procedure or fetched from the procedure through parameters.
- These parameters should be included in the calling statement.
- A Procedure in SQL can have a RETURN statement to return the control to the calling block, but it cannot return any values through the RETURN statement.

- Procedures cannot be called directly from SELECT statements. They can be called from another block or through EXEC keyword.

Syntax:

```
CREATE OR REPLACE PROCEDURE
<procedure_name>
        (
        <parameterl IN/OUT <datatype>
        ..
        .
        )
[ IS | AS ]
        <declaration_part>
BEGIN
        <execution part>
EXCEPTION
        <exception handling part>
END;
```

- CREATE PROCEDURE instructs the compiler to create new procedure in Oracle. Keyword 'OR REPLACE' instructs the compile to replace the existing procedure (if any) with the current one.
- Procedure name should be unique.
- Keyword 'IS' will be used, when the stored procedure in Oracle is nested into some other blocks. If the procedure is standalone then 'AS' will be used. Other than this coding standard, both have the same meaning.

Example1: Creating Procedure and calling it using EXEC

In this example, we are going to create an Oracle procedure that takes the name as input and prints the welcome message as output. We are going to use EXEC command to call procedure.

```
CREATE OR REPLACE PROCEDURE welcome_msg (p_name IN VARCHAR2)
IS
BEGIN
dbms_output.put_line ('Welcome '|| p_name);
END;
/
```

EXEC welcome_msg (‘Guru99’);

Code Explanation:

- **Code line 1**: Creating the procedure with name ‘welcome_msg’ and with one parameter ‘p_name’ of ‘IN’ type.
- **Code line 4**: Printing the welcome message by concatenating the input name.
- Procedure is compiled successfully.
- **Code line 7**: Calling the procedure using EXEC command with the parameter ‘Guru99’. Procedure is executed, and the message is printed out as “Welcome Guru99”.

What is Function?

Functions is a standalone PL/SQL subprogram. Like PL/SQL procedure, functions have a unique name by which it can be referred. These are stored as PL/SQL database objects. Below are some of the characteristics of functions.

- Functions are a standalone block that is mainly used for calculation purpose.
- Function use RETURN keyword to return the value, and the datatype of this is defined at the time of creation.
- A Function should either return a value or raise the exception, i.e. return is mandatory in functions.
- Function with no DML statements can be directly called in SELECT query whereas the function with DML operation can only be called from other PL/SQL blocks.
- It can have nested blocks, or it can be defined and nested inside the other blocks or packages.
- It contains declaration part (optional), execution part, exception handling part (optional).
- The values can be passed into the function or fetched from the procedure through the parameters.
- These parameters should be included in the calling statement.
- A PLSQL function can also return the value through OUT parameters other than using RETURN.
- Since it will always return the value, in calling statement it always accompanies with assignment operator to populate the variables.

Syntax:

```
CREATE OR REPLACE FUNCTION
<procedure_name>
        (
        <parameter1 IN/OUT <datatype>
        ..
        .
        )
        RETURN <datatype>
[ IS | AS ]
        <declaration_part>
BEGIN
        <execution part>
EXCEPTION
        <exception handling part>
END;
```

Syntax

```
CREATE OR REPLACE FUNCTION
<procedure_name>
(
<parameterl IN/OUT <datatype>
)
RETURN <datatype>
[ IS | AS ]
<declaration_part>
BEGIN
<execution part>
EXCEPTION
<exception handling part>
END;
```

- CREATE FUNCTION instructs the compiler to create a new function. Keyword 'OR REPLACE' instructs the compiler to replace the existing function (if any) with the current one.
- The Function name should be unique.
- RETURN datatype should be mentioned.
- Keyword 'IS' will be used, when the procedure is nested into some other blocks. If the procedure is standalone then 'AS' will be used. Other than this coding standard, both have the same meaning.

Example1: Creating Function and calling it using Anonymous Block

In this program, we are going to create a function that takes the name as input and returns the welcome message as output. We are going to use anonymous block and select statement to call the function.

```
1.  CREATE OR REPLACE FUNCTION welcome_msg_func ( p_name IN VARCHAR2)
2.  RETURN VARCHAR2
3.  IS
4.  BEGIN
5.  RETURN ('Welcome '|| p_name);
6.  END;
7.  /
```

Output:

Function created

Function Created

```
8.  DECLARE
9.  lv_msg VARCHAR2(250);
10. BEGIN
11. lv_msg := welcome_msg_func ('Guru99');
12. dbms_output.put_line(lv_msg);
13. END;
```

Calling function with 'Guru99' as parameter

Output:

Welcome Guru99

```
14. SELECT welcome_msg_func('Guru99') FROM DUAL;
```

Output:

Welcome Guru99

CREATE OR REPLACE FUNCTION welcome_msgJune (p_name IN VARCHAR2) RETURN VAR.CHAR2

IS

BEGIN

RETURN ('Welcome '|| p_name);

END;

/

DECLARE

lv_msg VARCHAR2(250);

BEGIN

lv_msg := welcome_msg_func ('Guru99');

dbms_output.put_line(lv_msg);

END;

SELECT welcome_msg_func('Guru99:) FROM DUAL;

Code Explanation:

- **Code line 1**: Creating the Oracle function with name 'welcome_msg_func' and with one parameter 'p_name' of 'IN' type.
- **Code line 2**: declaring the return type as VARCHAR2
- **Code line 5**: Returning the concatenated value 'Welcome' and the parameter value.
- **Code line 8**: Anonymous block to call the above function.
- **Code line 9**: Declaring the variable with datatype same as the return datatype of the function.
- **Code line 11**: Calling the function and populating the return value to the variable 'lv_msg'.
- **Code line 12**: Printing the variable value. The output you will get here is "Welcome Guru99"
- **Code line 14**: Calling the same function through SELECT statement. The return value is directed to the standard output directly.

Procedure Vs. Function: Key Differences

Procedure	**Function**
• Used mainly to a execute certain process	• Used mainly to perform some calculation
• Cannot call in SELECT statement	• A Function that contains no DML statements can be called in SELECT statement
• Use OUT parameter to return the value	• Use RETURN to return the value
• It is not mandatory to return the value	• It is mandatory to return the value
• RETURN will simply exit the control from subprogram.	• RETURN will exit the control from subprogram and also returns the value
• Return datatype will not be specified at the time of creation	• Return datatype is mandatory at the time of creation